📦 CATEGORY 1: Core Fundamentals & Architecture

> **Difficulty:** Beginner | **Topics:** 6 | **Terraform Interview Mastery Series**

Table of Contents

1. What is Terraform & IaC Philosophy
2. Terraform vs Other IaC Tools
3. Core Workflow: init → plan → apply → destroy
4. Terraform Architecture
5. HCL Syntax — Blocks, Arguments, Expressions
6. ⚠️ `.terraform.lock.hcl`

Topic 1: What is Terraform & IaC Philosophy

🔵 What It Is (Simple Terms)

Terraform is an open-source **Infrastructure as Code (IaC)** tool created by HashiCorp. It lets you **define, provision, and manage infrastructure** — servers, databases, networks, DNS records, Kubernetes clusters — using human-readable configuration files instead of clicking through a UI or running manual scripts.

IaC Philosophy means: *your infrastructure is code* — it lives in version control, it is reviewed, tested, and deployed the same way application code is.

🔵 Why It Exists — What Problem It Solves

Before IaC — The Dark Ages

Problem	Reality
Manual provisioning	Engineers clicked through AWS Console — slow, error-prone
Snowflake servers	Every server was slightly different — impossible to reproduce
No audit trail	Who created this S3 bucket? When? Why? Nobody knows
Environment drift	Dev, staging, prod diverged silently over time
Disaster recovery	Rebuilding infra after failure took days or weeks
Scaling	Adding 10 more servers = 10x the manual work

What IaC / Terraform Solves

- **Reproducibility** — Run the same config → get identical infrastructure every time

- **Version control** — Infrastructure changes go through Git, PRs, code review
- **Automation** — CI/CD pipelines can provision infra without human intervention
- **Documentation** — The code IS the documentation of what exists
- **Drift prevention** — Terraform detects when reality diverges from declared state
- **Disaster recovery** — Rebuild your entire infra by running `terraform apply`

🔵 How It Works Internally

Terraform follows a **declarative model** (not imperative):

```
Declarative:  "I want 3 EC2 instances of type t3.medium"
Imperative:   "Run this script: create instance 1, then create instance 2,
then..."
```

Terraform figures out **how** to get from the current state to your desired state. You describe the destination — Terraform plans the journey.

```
┌──────────────────────────────────────────────────────────┐
│                    IaC Execution Model                   │
│                                                          │
│  .tf files          Terraform Core         Real World    │
│  (Desired State) ──► Diff Engine      ──► AWS / GCP / Azure │
│                       ▲                                  │
│  .tfstate             │                                  │
│  (Current State) ─────┘                                  │
└──────────────────────────────────────────────────────────┘
```

🔵 Key Concepts Within IaC Philosophy

1. Declarative vs Imperative

```
# Declarative (Terraform) — describe WHAT you want
resource "aws_instance" "web" {
  ami           = "ami-0c55b159cbfafe1f0"
  instance_type = "t3.medium"
  count         = 3
}
```

```
# Imperative (Bash script) — describe HOW to do it
for i in 1 2 3; do
  aws ec2 run-instances --image-id ami-0c55b159cbfafe1f0 --instance-type
t3.medium
done
```

Why declarative wins: If 1 of 3 instances already exists, Terraform creates 2 more. The bash script creates 3 more, giving you 4 total — a bug.

2. Idempotency

Running Terraform 10 times on the same config produces the same result as running it once. No duplicates, no side effects.

3. Day 0 vs Day 1 vs Day 2

Day	Meaning	Terraform's Role
Day 0	Design & planning	Write configs
Day 1	Initial provisioning	`terraform apply`
Day 2	Ongoing operations	Updates, drift detection, scaling

Terraform excels at **Day 1 and Day 2** operations.

🔵 Short Interview Answer

> "Terraform is a declarative IaC tool that lets you define infrastructure in code and provision it consistently across any cloud. The core IaC philosophy is that infrastructure should be treated like application code — versioned, reviewed, and automated. Terraform specifically uses a declarative model where you describe the desired end state and it figures out how to get there, making it idempotent and reproducible."

🔵 Deep Dive Answer

> "Beyond the basics, what makes Terraform's IaC philosophy powerful is the combination of declarative syntax with state management. Terraform doesn't just execute commands — it builds a dependency graph of your resources (a DAG), computes the diff between your declared state and the current real-world state tracked in `terraform.tfstate`, and only makes the changes necessary to converge. This means it handles complex scenarios like: resource A must exist before resource B, or resource C needs to be replaced before resource D is updated. The IaC philosophy also extends to organizational practices — treating infrastructure changes as pull requests means you get peer review, automated testing, and an audit trail. At scale, this prevents the 'works on my laptop' problem for infrastructure — if it's in Git, it's reproducible."

🔵 Real World Production Example

```
# terraform/environments/prod/main.tf
# This single file provisions a production VPC with subnets
# Stored in Git — any change goes through PR review
```

```
terraform {
  required_version = ">= 1.5.0"
  backend "s3" {
    bucket = "mycompany-terraform-state"
    key    = "prod/vpc/terraform.tfstate"
    region = "us-east-1"
  }
}

module "vpc" {
  source  = "terraform-aws-modules/vpc/aws"
  version = "5.1.0"

  name = "prod-vpc"
  cidr = "10.0.0.0/16"

  azs             = ["us-east-1a", "us-east-1b", "us-east-1c"]
  private_subnets = ["10.0.1.0/24", "10.0.2.0/24", "10.0.3.0/24"]
  public_subnets  = ["10.0.101.0/24", "10.0.102.0/24", "10.0.103.0/24"]

  enable_nat_gateway = true
  single_nat_gateway = false  # HA: one per AZ in prod

  tags = {
    Environment = "production"
    ManagedBy   = "terraform"
    Team        = "platform"
  }
}
```

This file, in a Git repo, **is** your production network. Any engineer can read it and understand exactly what exists. Changes require a PR. CI runs `terraform plan` on every PR. The IaC philosophy in action.

🔵 Common Interview Questions

Q: What's the difference between declarative and imperative IaC?

> "Declarative means you describe the desired end state and the tool figures out how to get there — Terraform, CloudFormation. Imperative means you write the step-by-step instructions — Bash scripts, Ansible playbooks (though Ansible can be semi-declarative). Declarative is better for infrastructure because it handles idempotency automatically — if the resource already exists, Terraform won't recreate it."

Q: What is idempotency and why does it matter in IaC?

> "Idempotency means running the same operation multiple times produces the same result. In IaC, it means you can safely run `terraform apply` repeatedly without fear of creating duplicate resources or causing side effects. This is critical for CI/CD automation and disaster recovery — you don't need to worry about whether the apply has been run before."

Q: What are the advantages of IaC over manual provisioning?

> "Five key advantages: reproducibility (same config → same infra every time), version control (changes are audited and reviewable), automation (CI/CD can provision without humans), documentation (the code explains what exists and why), and disaster recovery (rebuild everything with one command). The hidden sixth advantage is drift detection — Terraform tells you when someone manually changed something outside of the code."

🔵 Gotchas & Edge Cases

- **IaC doesn't prevent all human error** — Terraform applies whatever you write. Writing `count = 0` deletes all your instances. The code must be reviewed.
- **Not everything should be in Terraform** — Transient resources, data pipelines, or things that change frequently might be better managed by application code or dedicated tools.
- **Terraform is eventually consistent** — After apply, some resources (like DNS propagation or IAM policy attachment) take time to actually be ready. Your code may need `depends_on` or `time_sleep` workarounds.
- **State is the source of truth, not your .tf files** — If state and reality diverge, Terraform goes by state. This is a common source of confusion for beginners.

🔵 Connections to Other Concepts

- → **Topic 3 (Workflow):** The IaC philosophy is realized through `init/plan/apply/destroy`
- → **Topic 4 (Architecture):** The declarative model is implemented via the Core + State engine
- → **Category 6 (State):** State is what enables Terraform to know current vs desired
- → **Category 8 (Security):** IaC enables security scanning before resources are created
- → **Category 9 (CI/CD):** IaC philosophy enables fully automated infra pipelines

---

Topic 2: Terraform vs Other IaC Tools

🔵 What It Is (Simple Terms)

There are several tools that can provision and manage infrastructure as code. Terraform is one of them, but it's important to know when Terraform is the right choice versus Pulumi, Ansible, AWS CloudFormation, or AWS CDK — and why.

🔵 The IaC Tool Landscape

```
┌──────────────────────────────────────────────────────────────────────────┐
│                      IaC Tool Categories                                 │
│                                                                          │
│  PROVISIONING tools          CONFIGURATION MANAGEMENT tools              │
```

```
│  (create infra)              (configure what's already there)      │
│                                                                    │
│  • Terraform  ◄── primary    • Ansible                             │
│  • Pulumi                    • Chef                                │
│  • CloudFormation            • Puppet                              │
│  • CDK                       • SaltStack                           │
└────────────────────────────────────────────────────────────────────┘
```

> ⚠ **Ansible can provision too** — but it's designed for configuration. Using it for provisioning is possible but painful at scale.

🔵 Head-to-Head Comparison

Terraform vs CloudFormation

Dimension	Terraform	CloudFormation
Cloud support	Multi-cloud (AWS, GCP, Azure, 3000+ providers)	AWS only
Language	HCL (purpose-built, readable)	JSON / YAML
State management	Explicit `.tfstate` file	Managed by AWS internally
Rollback	Manual — no automatic rollback	Built-in stack rollback
Modularity	Modules (very flexible)	Nested stacks (more rigid)
Community	Massive open source ecosystem	AWS-driven
Drift detection	`terraform plan` shows drift	Drift detection via CloudFormation console
Import existing resources	`terraform import`	CloudFormation import (limited)
Speed	Generally faster	Can be slow (AWS API polling)

When to use CloudFormation: AWS-only shops that want AWS-managed state and built-in rollback. Often used by orgs with strict AWS compliance requirements.

When to use Terraform: Multi-cloud, or when you need the flexibility, community modules, and expressiveness of HCL.

Terraform vs Pulumi

Dimension	Terraform	Pulumi
Language	HCL (DSL)	TypeScript, Python, Go, C#, Java
Learning curve	Low — HCL is simple	Higher — requires knowing a real language

Dimension	Terraform	Pulumi
Logic & loops	`for_each`, `count`, `dynamic`	Full language constructs
Testing	`terraform test`, Terratest	Native unit tests in your language
State	Local or remote backend	Pulumi Cloud or self-managed
Ecosystem	Massive — 3000+ providers	Growing — uses Terraform providers under the hood
Debugging	Limited	Full language debuggers

When to use Pulumi: Teams with strong software engineering background who want full programming language power. Complex logic that `for_each`/`count` can't express cleanly.

When to use Terraform: Most DevOps/Platform teams — simpler, larger community, more operators know it.

Terraform vs Ansible

Dimension	Terraform	Ansible
Primary purpose	Infrastructure provisioning	Configuration management
Model	Declarative	Mostly imperative (playbooks)
State	Tracks state explicitly	Stateless — runs tasks every time
Idempotency	Built-in	Module-dependent (some modules are idempotent, some aren't)
Server config	Not designed for it	Excellent at it
Cloud provisioning	Excellent	Possible but not ideal
Agent	Agentless	Agentless (SSH/WinRM)

Best practice: Use **both** — Terraform provisions the servers, Ansible configures them.

```
Terraform creates EC2 instance  →  Ansible installs nginx, configures app
```

Terraform vs AWS CDK

Dimension	Terraform	AWS CDK
Cloud support	Multi-cloud	AWS only (CDK for Terraform is separate)
Language	HCL	TypeScript, Python, Java, C#

Dimension	Terraform	AWS CDK
Abstraction level	Resource-level	Construct-level (higher abstraction)
Under the hood	Talks to APIs directly	Synthesizes to CloudFormation
Flexibility	Very high	High, but constrained by CF
For developers	Moderate	High — feels like writing app code

When to use CDK: Developer-led teams building AWS-native applications who prefer TypeScript/Python over HCL.

🔵 The Decision Matrix

```
Are you multi-cloud?
  YES → Terraform or Pulumi
  NO  → Could use CloudFormation/CDK if AWS-only

Do your team prefer writing real code (Python/TypeScript)?
  YES → Pulumi or CDK
  NO  → Terraform

Is your primary need server configuration (not provisioning)?
  YES → Ansible (+ Terraform for provisioning)
  NO  → Terraform

Are you in a large enterprise with existing CF investment?
  YES → Consider CloudFormation or CDK
  NO  → Terraform
```

🔵 Short Interview Answer

> "Terraform's main advantage over CloudFormation is multi-cloud support and a more expressive language. Versus Pulumi, Terraform uses HCL which has a lower barrier to entry for operators, while Pulumi suits teams with strong software engineering backgrounds. Ansible is a configuration management tool, not an infrastructure provisioner — they're complementary, not competitive. In most modern DevOps setups, Terraform provisions infra and Ansible handles OS-level configuration."

🔵 Real World Production Example

Common stack at a cloud-native startup:

```
Infrastructure Layer:     Terraform    (VPC, EKS, RDS, S3, IAM)
Configuration Layer:      Ansible      (OS packages, security hardening)
```

```
App Deployment:          Helm/ArgoCD  (Kubernetes manifests)
Policy Enforcement:      Sentinel      (Terraform Cloud)
```

🔵 Common Interview Questions

Q: Why would you choose Terraform over CloudFormation?

> "Terraform is cloud-agnostic — I can manage AWS, GCP, Azure, Cloudflare, and Datadog resources in the same codebase. HCL is more readable than YAML/JSON CloudFormation. The module ecosystem is richer. And `terraform plan` gives a very clear preview of changes before applying. The tradeoff is that state management is your responsibility, while CloudFormation manages it for you."

Q: Can Terraform replace Ansible?

> "No — they solve different problems. Terraform provisions infrastructure (creates EC2 instances, VPCs, databases). Ansible configures what's running on that infrastructure (installs packages, manages config files, handles OS-level tasks). The idiomatic pattern is to use both: Terraform creates the resource, an Ansible playbook or cloud-init script configures it."

Q: Have you used Pulumi? How does it compare?

> "Pulumi lets you use real programming languages which enables more sophisticated logic. But for most infrastructure teams, HCL's simplicity is a feature, not a bug. Terraform's ecosystem is also far larger — there are thousands of community modules. I'd choose Pulumi if the team had strong TypeScript/Python skills and needed complex logic that Terraform's meta-arguments couldn't express cleanly."

🔵 Gotchas & Edge Cases

- ⚠ **CDK for Terraform (CDKTF)** exists — it lets you write Terraform using TypeScript/Python but synthesizes to HCL. Interviewers sometimes ask about this to see if you're current.
- ⚠ **Pulumi uses Terraform providers under the hood** — many Pulumi providers are bridges over Terraform providers. This is often a surprise.
- **Terraform is NOT a configuration management tool** — trying to use it to install software on servers (`remote-exec` provisioner) is an anti-pattern. Use it only for provisioning.
- **Ansible is not idempotent by default** — some Ansible modules check current state, others don't. This surprises engineers who expect Ansible to behave like Terraform.

🔵 Connections to Other Concepts

- → **Topic 1 (IaC Philosophy):** Understanding why IaC tools exist in the first place
- → **Category 9 (CI/CD):** How Terraform integrates into pipelines vs other tools
- → **Category 10 (Advanced — Terragrunt):** Terragrunt as a Terraform wrapper to address its limitations

---

Topic 3: Core Workflow — `init` → `plan` → `apply` → `destroy`

🔵 What It Is (Simple Terms)

Every Terraform operation follows a four-command lifecycle. Understanding what each command does internally is foundational — and interviewers test this deeply.

```
terraform init     →  Download providers & modules, set up backend
terraform plan     →  Preview what will change (dry run)
terraform apply    →  Execute the changes against real infrastructure
terraform destroy  →  Tear down all managed resources
```

🔵 Why It Exists — What Problem It Solves

The workflow separates **setup**, **planning**, and **execution** deliberately:

- `init` ensures all dependencies are present before anything runs
- `plan` allows humans (and CI systems) to **review changes before they happen** — a critical safety gate
- `apply` makes it real — but only after plan is approved
- `destroy` is explicit, intentional, and separate — you can't accidentally destroy by running `apply`

🔵 How Each Command Works Internally

`terraform init`

```
What it does:
1. Reads terraform {} block in .tf files
2. Downloads required providers from registry.terraform.io
   (or configured mirror) into .terraform/providers/
3. Downloads modules from sources into .terraform/modules/
4. Initializes the configured backend (S3, GCS, Terraform Cloud)
5. Creates/updates .terraform.lock.hcl
```

```
terraform init

# Common flags:
terraform init -upgrade          # Upgrade providers to latest allowed
version
terraform init -backend=false    # Skip backend initialization
```

```
terraform init -reconfigure      # Force reconfiguration of backend
terraform init -migrate-state    # Migrate state to new backend
```

Directory structure after init:

```
.
├── main.tf
├── variables.tf
├── outputs.tf
├── .terraform/
│   ├── providers/
│   │   └── registry.terraform.io/hashicorp/aws/5.0.0/linux_amd64/
│   └── modules/
│       └── vpc/
├── .terraform.lock.hcl     ← provider version lock file
└── terraform.tfstate        ← state file (if local backend)
```

---

terraform plan

```
What it does internally:
1. Reads all .tf configuration files
2. Reads current state from backend (terraform.tfstate)
3. Calls provider APIs to refresh resource states (unless -refresh=false)
4. Builds dependency graph (DAG) of all resources
5. Computes diff: desired state vs current state
6. Outputs a detailed execution plan showing +/~/- changes
```

```
terraform plan

# Common flags:
terraform plan -out=tfplan          # Save plan to file (use in CI/CD)
terraform plan -var="env=prod"      # Pass variable inline
terraform plan -var-file="prod.tfvars"
terraform plan -target=aws_instance.web  # Plan only specific resource
terraform plan -refresh=false       # Skip API refresh (faster, riskier)
terraform plan -destroy             # Show what destroy would do
```

Reading plan output:

```
Terraform will perform the following actions:

  # aws_instance.web will be created
  + resource "aws_instance" "web" {        ← + = create
      + ami           = "ami-0c55b159cbfafe1f0"
```

```
      + instance_type = "t3.medium"
    }

  # aws_security_group.web will be updated in-place
  ~ resource "aws_security_group" "web" {  ← ~ = update (in-place)
      ~ description = "old" -> "new"
    }

  # aws_instance.old will be destroyed
  - resource "aws_instance" "old" {        ← - = destroy
    }

  # aws_db_instance.main must be replaced
-/+ resource "aws_db_instance" "main" {   ← -/+ = destroy then create
      ~ engine_version = "13.4" -> "14.0" # forces replacement
    }

Plan: 1 to add, 1 to change, 1 to destroy.
```

---

terraform apply

```
What it does internally:
1. (Optionally) runs plan again if no saved plan file
2. Shows plan output and prompts for confirmation (unless -auto-approve)
3. Acquires state lock (prevents concurrent applies)
4. Walks the dependency graph in parallel (default 10 concurrent
operations)
5. Creates/updates/destroys resources via provider API calls
6. Updates state file after each successful resource operation
7. Releases state lock
```

```
terraform apply                       # Interactive — shows plan, asks
yes/no
terraform apply -auto-approve         # No confirmation prompt (CI/CD)
terraform apply tfplan                # Apply a saved plan file
terraform apply -parallelism=20       # Increase concurrent API calls
terraform apply -var="instance_count=5"
```

> ⚠ **Critical:** Always use `terraform apply tfplan` in CI/CD — apply the same plan that was reviewed, not a new plan.

---

terraform destroy

```
What it does internally:
1. Creates a destroy plan (all resources marked for deletion)
```

```
2. Prompts for confirmation
3. Destroys resources in reverse dependency order
4. Removes entries from state file
```

```
terraform destroy                          # Destroy everything
terraform destroy -target=aws_instance.web # Destroy specific resource
terraform destroy -auto-approve            # No confirmation (dangerous!)
```

🔵 The Full Workflow Diagram

```
┌──────────────────────────────────────────────────────────────────────┐
│                    Terraform Core Workflow                           │
│                                                                      │
│  Write .tf files                                                     │
│       │                                                              │
│       ▼                                                              │
│  terraform init ──► Downloads providers/modules, inits backend      │
│       │                                                              │
│       ▼                                                              │
│  terraform plan ──► Reads state + .tf files → computes diff         │
│       │              Shows +/-/~ preview                             │
│       │                                                              │
│       ▼  (human/CI reviews plan)                                     │
│                                                                      │
│  terraform apply ──► Acquires lock → executes plan → updates state  │
│       │                                                              │
│       ▼  (when tearing down)                                         │
│                                                                      │
│  terraform destroy ──► Reverse apply, deletes all resources         │
└──────────────────────────────────────────────────────────────────────┘
```

🔵 CI/CD Workflow Pattern

```
# In a CI/CD pipeline (GitHub Actions / GitLab CI)

# Step 1: On PR open
terraform init
terraform validate
terraform plan -out=tfplan    # Save plan artifact

# Step 2: Human reviews plan output in PR

# Step 3: On PR merge to main
terraform apply tfplan         # Apply the EXACT plan that was reviewed
```

🔵 Short Interview Answer

> "`init` downloads providers and modules and sets up the backend. `plan` is a dry run — it reads your config and state, calls provider APIs to check current reality, builds a diff, and shows you exactly what will change. `apply` executes that plan, acquires a state lock, makes API calls in parallel, and updates state. `destroy` is a reverse apply — it deletes resources in reverse dependency order. In CI/CD, you always save the plan with `-out` and apply that exact file to ensure what you reviewed is what gets applied."

🔵 Common Interview Questions

Q: What happens if you run `apply` without `plan` first?

> "Terraform runs a plan internally before applying and shows it to you — then asks for confirmation unless you pass `-auto-approve`. It's the same plan process, just combined into one step. In production, best practice is to always use `plan -out=tfplan` and then `apply tfplan` so the plan is an artifact that can be reviewed and stored."

Q: What does `terraform init` do to my filesystem?

> "It creates a `.terraform` directory containing downloaded provider binaries and module source code. It also creates or updates `.terraform.lock.hcl` which pins the exact provider versions and their checksums. The `.terraform` directory should be in `.gitignore` — it's environment-specific and regenerated by `init`. The lock file, however, should be committed to Git."

Q: Can `plan` output ever be wrong — i.e., can apply do something different than plan showed?

> "Yes, in rare cases. If infrastructure changes between `plan` and `apply` (someone manually changed something, or an async operation completed), the apply may behave differently than the plan showed. This is why using saved plan files (`-out`) with a short window between plan and apply is best practice. It's also why `terraform refresh` and `-refresh=false` matter — the plan is only as accurate as the state refresh."

🔵 Gotchas & Edge Cases

- ⚠ **`-auto-approve` in production is dangerous** — always gate apply with a human approval step or Sentinel policy in CI/CD
- ⚠ **`-target` is a last resort** — targeting specific resources creates state inconsistencies and should only be used for emergencies, not routine operations
- **`terraform destroy` destroys everything in state** — not everything in the directory. If resources were removed from `.tf` files but remain in state, they will be destroyed
- **Plan files are binary and environment-specific** — a plan file from one machine may not be applicable on another if provider versions differ
- **`init` must be re-run after** — changing backend config, adding new providers, or updating module sources

---

🔵 Connections to Other Concepts

- → **Topic 4 (Architecture):** Understand what Core, Provider, and State do during each workflow step
- → **Category 6 (State):** `plan` and `apply` read/write state — state management is central to the workflow
- → **Category 9 (CI/CD):** The workflow is what CI/CD pipelines automate
- → **Category 10 (Troubleshooting):** Most Terraform problems happen during `plan` or `apply`

Topic 4: Terraform Architecture — CLI, Core, Providers, State

---

🔵 What It Is (Simple Terms)

Terraform has four major components that work together every time you run a command. Understanding their roles explains why Terraform behaves the way it does.

```
CLI        →  What you type in your terminal
Core       →  The brain — reads config, builds graph, computes diff
Providers  →  The hands — makes API calls to AWS, GCP, Azure, etc.
State      →  The memory — tracks what currently exists
```

🔵 Architecture Diagram

```
┌──────────────────────────────────────────────────────────────────────┐
│                    Terraform Architecture                            │
│                                                                      │
│   ┌──────────┐   commands   ┌──────────────────────────────────┐    │
│   │          │ ────────────► │          Terraform Core          │    │
│   │   CLI    │               │                                  │    │
│   │          │ ◄──────────── │  • Config Parser (HCL)           │    │
│   └──────────┘   output      │  • Resource Graph Builder (DAG)  │    │
│                              │  • Plan Engine (diff)            │    │
│                              │  • Apply Engine (executor)       │    │
│                              └──────────────────────────────────┘    │
│                                          │ Plugin Protocol (gRPC)    │
│                              ┌───────────▼──────────────────────┐    │
│                              │           Providers              │    │
│                              │                                  │    │
│                              │  provider "aws" { ... }          │    │
│                              │  provider "google" { ... }       │    │
│                              │  provider "kubernetes" { ... }   │    │
│                              └──────────────────────────────────┘    │
│                                          │ API calls                 │
```

```
│                                  ┌──────────────▼──────────────────────────┐   │
│                                  │     Real Infrastructure                 │   │
│                                  │   AWS / GCP / Azure / Cloudflare        │   │
│                                  └─────────────────────────────────────────┘   │
│                                                                                 │
│                                  ┌─────────────────────────────────────────┐   │
│                                  │           State Backend                 │   │
│                                  │  terraform.tfstate (S3, GCS, TFC)       │   │
│                                  └─────────────────────────────────────────┘   │
└─────────────────────────────────────────────────────────────────────────────────┘
```

🔵 Each Component in Detail

1. CLI (Command Line Interface)

The CLI is the user-facing layer. It parses your commands (`init`, `plan`, `apply`) and delegates to Core.

```
terraform plan     # CLI parses this, hands off to Core
terraform apply    # CLI shows output, handles interactive prompts
terraform fmt      # CLI formats .tf files (no Core needed)
```

The CLI binary is a **single Go binary** — no separate install needed for providers.

---

2. Terraform Core

The brain of Terraform. Written in Go. Responsible for:

```
1. Reading and parsing .tf configuration files (HCL parser)
2. Loading state from backend
3. Calling providers to refresh resource states
4. Building the Resource Graph (DAG - Directed Acyclic Graph)
5. Computing the diff (plan)
6. Walking the graph and calling provider CRUD operations (apply)
7. Writing updated state back to backend
```

The DAG is key: Core builds a dependency graph where nodes are resources and edges are dependencies. It walks this graph in parallel (up to `parallelism` simultaneous operations), respecting dependencies.

```
aws_vpc.main ──► aws_subnet.private ──► aws_instance.web
                                     └──► aws_instance.app
```

`aws_instance.web` and `aws_instance.app` can be created in parallel because they both only depend on `aws_subnet.private`.

---

3. Providers

Providers are **plugins** — separate binaries that Terraform Core communicates with via **gRPC** (using the Terraform Plugin Protocol).

Each provider:

- Knows how to authenticate with a specific cloud/API
- Defines which resources and data sources it supports
- Translates Terraform's CRUD operations into real API calls
- Validates resource configurations

```
# Provider configuration
provider "aws" {
  region  = "us-east-1"
  profile = "production"
}

# Provider declaration
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}
```

```
Provider lifecycle:
terraform init  →  Downloads provider binary to .terraform/providers/
terraform plan  →  Core starts provider as subprocess, communicates via
gRPC
                   Core asks: "What is the current state of
aws_instance.web?"
                   Provider calls AWS API, returns JSON response
terraform apply →  Core tells provider: "Create this resource with these
params"
                   Provider calls AWS CreateInstance API, returns resource
ID
                   Core writes resource ID and attributes to state
```

---

4. State

State is Terraform's memory — it maps your configuration to real-world resources.

```
// Simplified terraform.tfstate
{
```

```
  "version": 4,
  "terraform_version": "1.5.0",
  "resources": [
    {
      "type": "aws_instance",
      "name": "web",
      "provider": "provider[\"registry.terraform.io/hashicorp/aws\"]",
      "instances": [
        {
          "attributes": {
            "id": "i-0a1b2c3d4e5f67890",
            "ami": "ami-0c55b159cbfafe1f0",
            "instance_type": "t3.medium",
            "public_ip": "54.12.34.56"
          }
        }
      ]
    }
  ]
}
```

Why state is needed:

- Without state, Terraform can't know `aws_instance.web` corresponds to `i-0a1b2c3d4e5f67890`
- Without state, every `plan` would show "create everything" because Terraform wouldn't know what already exists
- State enables `plan` to diff desired vs actual

🔵 Short Interview Answer

> "Terraform has four components: the CLI which handles user commands and output, Core which is the brain — it parses HCL, builds a dependency graph, computes diffs, and orchestrates execution, Providers which are separate plugins that translate Terraform operations into real API calls for AWS/GCP/Azure, and State which is the memory that maps configuration to real resource IDs. Core and Providers communicate via gRPC using the Terraform Plugin Protocol."

🔵 Common Interview Questions

Q: What is the Terraform Plugin Protocol?

> "Providers are separate processes — not linked into the Terraform binary. Core starts providers as subprocesses and communicates with them via gRPC using a defined interface called the Plugin Protocol (currently Plugin SDK v2 or the newer Plugin Framework). This design means providers can be written by anyone and updated independently of Terraform Core."

Q: What's the difference between Terraform Core and a provider?

> "Core is the orchestration layer — it reads config, builds the dependency graph, and decides what needs to change. Providers are the execution layer — they actually make the API calls. Core tells the

> AWS provider 'create an EC2 instance with these attributes' — the provider figures out which AWS API to call and handles the auth. This separation lets you use the same Core for any cloud by swapping providers."

🔵 Gotchas & Edge Cases

- ⚠ **Provider versions are independent of Terraform versions** — a new AWS provider might not work with an old Terraform core. Always test upgrades.
- **Providers run as separate OS processes** — provider crashes don't crash Terraform Core, but you'll see an error and the operation will fail
- **The DAG can have cycles** — Terraform detects and errors on circular dependencies (e.g., resource A depends on resource B which depends on resource A). This is a common source of confusion.
- **State is written per-resource, not atomically** — if apply fails halfway, state reflects what was successfully applied. Partially-applied state is a real scenario to handle.

🔵 Connections to Other Concepts

- → **Topic 2 (Providers):** Provider architecture deep dive
- → **Category 6 (State):** State management is half the architecture
- → **Category 10 (Troubleshooting):** Cycle errors, provider crashes are architecture-level issues

---

Topic 5: HCL Syntax — Blocks, Arguments, Expressions

🔵 What It Is (Simple Terms)

HCL (HashiCorp Configuration Language) is the language you write Terraform configs in. It's purpose-built to be readable by both humans and machines. Understanding its core building blocks is essential for writing clean, correct Terraform.

🔵 Core Building Blocks

1. Blocks

A block has a **type**, optional **labels**, and a **body** containing arguments.

```
# Syntax: <block_type> "<label1>" "<label2>" { body }

resource "aws_instance" "web" {   # block_type="resource",
labels="aws_instance","web"
  ami           = "ami-12345"     # argument
  instance_type = "t3.medium"     # argument
```

```
}

# Block types you'll use constantly:
# resource, data, module, variable, output, locals, terraform, provider
```

---

2. Arguments

Arguments assign values within blocks.

```
resource "aws_s3_bucket" "example" {
  bucket = "my-unique-bucket-name"    # string argument
  force_destroy = true                # boolean argument

  tags = {                            # map argument
    Name        = "My Bucket"
    Environment = "Production"
  }
}
```

---

3. Nested Blocks

Blocks can contain other blocks.

```
resource "aws_security_group" "web" {
  name   = "web-sg"
  vpc_id = aws_vpc.main.id

  ingress {                     # nested block — no quotes, no labels
    from_port   = 443
    to_port     = 443
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}
```

---

4. Expressions

Expressions produce values. The key types:

```
# Reference expressions — access other resources/variables
resource "aws_subnet" "private" {
  vpc_id     = aws_vpc.main.id        # resource reference
  cidr_block = var.subnet_cidr        # variable reference
  tags       = local.common_tags      # local reference
}

# String interpolation
resource "aws_instance" "web" {
  tags = {
    Name = "${var.env}-web-server"    # interpolation
  }
}

# Conditional expression
resource "aws_instance" "web" {
  instance_type = var.env == "prod" ? "t3.large" : "t3.micro"
}

# For expression — transform lists/maps
locals {
  upper_names = [for name in var.names : upper(name)]
  filtered    = [for s in var.services : s if s.enabled]
  name_map    = {for s in var.services : s.name => s.port}
}

# Splat expression — shorthand for attribute access across lists
output "instance_ids" {
  value = aws_instance.web[*].id     # collect id from all instances
}
```

---

5. Data Types

```
# Primitive types
variable "name"    { type = string  }   # "hello"
variable "count"   { type = number  }   # 42 or 3.14
variable "enabled" { type = bool    }   # true or false

# Collection types
variable "zones" {
  type    = list(string)
  default = ["us-east-1a", "us-east-1b"]
}

variable "tags" {
  type    = map(string)
  default = { env = "prod", team = "platform" }
}
```

```
variable "servers" {
  type = set(string)              # like list but unordered and unique
  default = ["web", "api", "db"]
}

# Structural types
variable "config" {
  type = object({
    instance_type = string
    count         = number
    enabled       = bool
  })
}

variable "coordinates" {
  type = tuple([string, number, bool])  # fixed-length, mixed types
}

# any — Terraform infers type
variable "flexible" { type = any }
```

---

6. Key Block Types Reference

```
# terraform block — configure Terraform itself
terraform {
  required_version = ">= 1.5.0"
  required_providers {
    aws = { source = "hashicorp/aws", version = "~> 5.0" }
  }
  backend "s3" { ... }
}

# provider block — configure a provider
provider "aws" {
  region = "us-east-1"
}

# resource block — declare infrastructure
resource "aws_vpc" "main" {
  cidr_block = "10.0.0.0/16"
}

# data block — read existing infrastructure
data "aws_ami" "ubuntu" {
  most_recent = true
  filter { name = "name", values = ["ubuntu/images/*22.04*"] }
}

# variable block — input parameters
variable "environment" {
```

```
  type        = string
  description = "Deployment environment"
  default     = "dev"
}

# output block — export values
output "vpc_id" {
  value       = aws_vpc.main.id
  description = "The VPC ID"
}

# locals block — computed local values
locals {
  common_tags = {
    ManagedBy   = "terraform"
    Environment = var.environment
  }
}
```

🔵 Short Interview Answer

> "HCL has three core constructs: blocks which are containers with a type, optional labels, and a body; arguments which assign values inside blocks; and expressions which produce values — including resource references, string interpolation, conditionals, and for-expressions. Types in Terraform include primitives (string, number, bool), collections (list, map, set), and structural types (object, tuple). The key block types are terraform, provider, resource, data, variable, output, and locals."

🔵 Common Interview Questions

Q: What is the difference between a list and a set in Terraform?

> "A list is ordered and allows duplicates — `["a", "b", "a"]`. A set is unordered and unique — `["a", "b"]`. Sets are important for `for_each` because `for_each` requires a set or map. If you pass a list to `for_each`, you must convert it with `toset()`. Sets don't support index access like `var.list[0]` because they have no defined order."

Q: What is the difference between `object` and `map` types?

> "A `map` is a collection of values all of the same type — `map(string)` means every value is a string. An `object` is a structural type with named attributes that can have different types — like a struct in Go or a typed dict. Use `map` when you have a variable number of same-typed key-value pairs. Use `object` when you need a fixed schema with mixed types."

🔵 Gotchas & Edge Cases

- ⚠ **`null` is a valid value** — assigning `null` to an argument tells Terraform to use the provider's default for that attribute

- **String interpolation `"${}"` isn't always needed** — `vpc_id = aws_vpc.main.id` is cleaner than `vpc_id = "${aws_vpc.main.id}"`
- **The `any` type defers type checking** — useful for module flexibility but loses compile-time safety
- **Tuple vs List** — Terraform sometimes returns tuples (fixed-length, mixed-type) from expressions where you expect lists. Using `tolist()` can convert when needed
- **Block labels must be unique** — two `resource "aws_instance" "web"` blocks in the same module will error

---

🔵 Connections to Other Concepts

- → **Category 4 (Variables/Expressions):** HCL syntax is the foundation for all variable and expression work
- → **Category 5 (Meta-Arguments):** `count`, `for_each`, `lifecycle` are all HCL block constructs
- → **Category 7 (Modules):** Module inputs/outputs are HCL variable and output blocks

---

Topic 6: ⚠ `.terraform.lock.hcl` — Why It Exists and What It Locks

🔵 What It Is (Simple Terms)

`.terraform.lock.hcl` is a **dependency lock file** for Terraform providers. It records the exact version and cryptographic checksums of every provider used in your configuration. Think of it like `package-lock.json` in Node.js or `Pipfile.lock` in Python.

> ⚠ This is a **heavily tested interview topic** — most candidates know it exists but can't explain what it actually locks, why checksums matter, or when to commit vs ignore it.

🔵 Why It Exists — What Problem It Solves

The problem without a lock file:

```
required_providers {
  aws = {
    source  = "hashicorp/aws"
    version = "~> 5.0"           # This matches 5.0, 5.1, 5.2, 5.99...
  }
}
```

If you run `terraform init` today you get `5.1.0`. Your teammate runs it next month and gets `5.3.0`. The provider behavior may differ. Your CI runs `5.1.0`, production runs `5.3.0`. **You have a reproducibility problem.**

The lock file solves this by recording the exact version installed so every `init` uses the same version.

🔵 What's Inside the Lock File

```
# .terraform.lock.hcl

provider "registry.terraform.io/hashicorp/aws" {
  version     = "5.1.0"                     # Exact version pinned
  constraints = "~> 5.0"                    # Original version constraint
  hashes = [
    "h1:abc123...",                          # Hash of provider binary
    "zh:def456...",                          # zh = ziphash (of provider
zip)
    "zh:ghi789...",
  ]
}

provider "registry.terraform.io/hashicorp/random" {
  version     = "3.5.1"
  constraints = ">= 3.0.0"
  hashes = [
    "h1:xyz321...",
    "zh:uvw654...",
  ]
}
```

Three things locked:

1. **Exact version** — `5.1.0`, not `~> 5.0`
2. **Version constraints** — the original rule that was satisfied
3. **Checksums (hashes)** — cryptographic fingerprints of the provider binary for multiple platforms

---

🔵 The Hash Types

```
h1:  →  Hash of the extracted provider binary (for the current OS/arch)
zh:  →  Hash of the provider zip file (platform-independent)
```

Multiple hashes exist because the same provider version has different binaries for linux_amd64, darwin_arm64, windows_amd64, etc. The lock file stores hashes for **all platforms** your team uses.

🔵 How It Interacts with `terraform init`

```
First run (no lock file):
  terraform init
```

```
    → Downloads latest version matching constraints
    → Creates .terraform.lock.hcl with exact version + hashes

  Subsequent runs (lock file exists):
    terraform init
    → Reads lock file
    → Downloads EXACT version from lock file
    → Verifies downloaded binary matches stored hash
    → Fails if hash doesn't match (supply chain attack protection)

  Upgrade:
    terraform init -upgrade
    → Ignores lock file
    → Downloads latest version matching constraints
    → UPDATES the lock file with new version + hashes
```

🔵 Should You Commit It to Git?

Yes. Always. No exceptions.

```
✅ Commit .terraform.lock.hcl   →  Reproducible builds, security, team
consistency
❌ .gitignore .terraform.lock.hcl →  Different providers on every
machine/CI run
```

The `.terraform/` directory goes in `.gitignore`. The lock file does NOT.

```
# .gitignore
.terraform/          # provider binaries, module downloads — regenerated
by init
*.tfstate            # state files — in remote backend, never in Git
*.tfstate.backup
.terraform.tfvars    # sensitive vars — never in Git

# DO NOT ignore:
# .terraform.lock.hcl  ← must be in Git
```

🔵 Adding Hashes for Multiple Platforms

If your team uses Mac (darwin_arm64) and CI uses Linux (linux_amd64), the lock file may only contain hashes for your local platform. Add hashes for other platforms with:

```
terraform providers lock \
  -platform=linux_amd64 \
```

```
  -platform=darwin_arm64 \
  -platform=windows_amd64
```

This fetches and stores hashes for all specified platforms so CI and local dev environments all validate correctly.

🔵 Short Interview Answer

> "`.terraform.lock.hcl` is Terraform's provider dependency lock file — like `package-lock.json` for Node. It records the exact provider version and cryptographic checksums for each provider after `terraform init`. This ensures every engineer and every CI run uses the same provider version rather than resolving the version constraint fresh each time. It should always be committed to Git. Upgrading providers requires `terraform init -upgrade` which updates the lock file."

🔵 Deep Dive Answer

> "The lock file solves two problems: reproducibility and supply chain security. On reproducibility: version constraints like `~> 5.0` can match many versions — the lock file pins the exact version so builds are deterministic. On security: the lock file stores cryptographic hashes of the provider binary. If a provider binary is tampered with (supply chain attack), `terraform init` will detect the hash mismatch and fail before executing anything. There are two hash types — `h1` hashes the extracted binary for the current platform, and `zh` hashes the zip file (platform-independent). For teams using multiple OS/architectures, you need to run `terraform providers lock` with `-platform` flags to populate hashes for all platforms, otherwise CI might fail hash verification because it's on a different OS than your local machine."

🔵 Real World Production Example

```
# Common issue: CI fails with hash mismatch
# Error: "the current package for ... doesn't match any of the checksums"

# Root cause: developer on Mac M1 (darwin_arm64) ran init locally
# Lock file only has darwin_arm64 hashes
# CI runs on linux_amd64 — hash not in lock file

# Fix:
terraform providers lock \
  -platform=linux_amd64 \
  -platform=darwin_arm64

git add .terraform.lock.hcl
git commit -m "Add linux_amd64 provider hashes for CI"
```

🔵 Common Interview Questions

Q: What's the difference between `.terraform.lock.hcl` and `required_providers` version constraints?

> "`required_providers` sets the version constraint — the range of acceptable versions, like `~> 5.0`. The lock file records which specific version within that range was actually selected and installed — like `5.1.0`. The constraint is the rule, the lock file is the resolution of that rule. Once the lock file exists, `init` uses the locked version, not the latest matching the constraint."

Q: What happens if you delete the lock file?

> "The next `terraform init` will resolve the version constraints fresh and potentially install a newer provider version. This is sometimes intentional when you want to upgrade. But it creates a risk — if the newer version has breaking changes or bugs, your infrastructure management could be affected. Always review lock file changes in code review, just like dependency updates in application code."

Q: Can the lock file protect against supply chain attacks?

> "Partially. The lock file stores cryptographic hashes of provider binaries. If a provider binary is replaced with a malicious version (but keeps the same version number), `terraform init` will detect the hash mismatch and refuse to use the binary. This is an important security control. However, it only protects against tampering with already-locked versions — it doesn't protect against a malicious provider being published under a new version number."

🔵 Gotchas & Edge Cases

- ⚠ **Lock file only locks providers, not modules** — module versions are pinned in the `source` argument with a `version` parameter. There is no separate lock file for modules.
- ⚠ **`-upgrade` flag updates the lock file** — this is intentional but can surprise people. Running `init -upgrade` in CI without reviewing the lock file diff is dangerous.
- **Multi-platform CI/CD** — if local dev is Mac and CI is Linux, you'll hit hash mismatch errors until you run `terraform providers lock` with both platforms
- **Air-gapped environments** — in environments without internet access, you need a provider mirror. The lock file hashes must be populated from the mirror, not the public registry.
- **Lock file merge conflicts** — when multiple engineers update providers simultaneously, the lock file can have merge conflicts. Always resolve these carefully — don't just pick one side blindly.

🔵 Connections to Other Concepts

- → **Topic 2 (Providers):** Lock file is how provider versions are pinned
- → **Category 8 (Security):** Hash verification is a supply chain security control
- → **Category 9 (CI/CD):** CI failures due to missing platform hashes is a common real-world problem
- → **Category 10 (Troubleshooting):** Hash mismatch errors are a common `init` failure mode

---

📊 Category 1 Summary — Quick Reference Card

Topic	One-Line Summary	Interview Weight
1. IaC Philosophy	Declarative, idempotent, version-controlled infra	⭐⭐⭐
2. Terraform vs Others	Multi-cloud, HCL, community — vs CF/Pulumi/Ansible	⭐⭐⭐⭐
3. Core Workflow	init/plan/apply/destroy — know each step internally	⭐⭐⭐⭐⭐
4. Architecture	CLI+Core+Providers+State — gRPC plugin protocol	⭐⭐⭐⭐
5. HCL Syntax	Blocks, arguments, expressions, types	⭐⭐⭐
6. Lock File ⚠	Pins provider versions + checksums — always commit	⭐⭐⭐⭐

🎯 Category 1 — Top 5 Interview Questions to Master

1. **"Walk me through what happens when you run `terraform plan`"** — describe Core reading config, refreshing state, building DAG, computing diff
2. **"What is the difference between Terraform and Ansible?"** — provisioning vs configuration management, declarative vs imperative
3. **"Should `.terraform.lock.hcl` be committed to Git? Why?"** — yes, reproducibility + supply chain security
4. **"What is a provider in Terraform?"** — plugin, gRPC, translates to API calls
5. **"What does `terraform init` do?"** — downloads providers/modules, inits backend, creates lock file

---

> **Next:** Category 2 — Providers (Topics 7–11) Type `Category 2` to continue, `quiz me` to be tested on Category 1, or `deeper` on any specific topic.